

A Project Report

on

WorkNest

By

Rahul Shinde ,

Aditya Ghorad

MCA – I, SEM – I

2025-26

To

Savitribai Phule Pune University,Pune

In Partial Fulfillment of the Degree of
Master in Computer Application (M. C. A.)

Under The Guidance Of

Prof. Omkar Kumbhar

Suryadatta Group of Institutes, Pune

Suryadatta Institute of Business Management And Technology (SIBMT)

Date:-

CERTIFICATE

This is to certify that **Mr. Rahul Shinde and Aditya Ghorad** , has successfully completed project work entitled **“WorkNest”** in partial fulfillment of MCA – I Semester-I program for the year A.Y. 2025-26. She have worked under our guidance and direction.

Prof. Omkar Kumbhar
Guide

Dr.Manisha Kumbhar Project
PROFESSOR & DIRECTOR

Examiner 1

Examiner 2

Date : Place :

Acknowledgment

We are the student of MCA first year. Here by we express our thanks to our project guide for allowing us to do the project on Worknest. This project work has been the most exciting part of our learning experience which would be an asset for our future carrier. We would especially like to thank our guide and mentor Prof. Omkar Kumbhar , who constantly guided us in developing, pushing us to search for more answers to her numerous questions. Also I would like to thank Dr. Rupali Dahake & Prof. Apurva Patil, project coordinators for their support. As a building block of MCA Department, I thank Dr. Manisha Kumbhar, PROFESSOR & DIRECTOR MCA Department for her continuous support and help. We are grateful to many classmates who contributed their suggestions. Their hard work and examples push us to limits of our capability and encourage us daily.

Thank You

Student Name : Rahul Shinde and Aditya Ghorad

Index

Chapter	Page number
CHAPTER 1: INTRODUCTION	
1.1 Existing System	
1.2 Need for System	
1.3 Operating Environment Hardware and Software	
CHAPTER 2: PROPOSED SYSTEM	
2.1 Proposed System (Introduction of system)	
2.2 Module specifications (Scope)	
2.3 Objectives of System	
CHAPTER 3: ANALYSIS & DESIGN	
3.1 Use Case Diagrams	
3.2 Activity Diagram	
3.3 Class Diagram	
3.4 Module Hierarchy Diagram	
3.5 Table specifications (Database design)	
3.6 Data dictionary	
CHAPTER 4: USER MANUAL	
4.1 User Interface Screens (Input)	
4.2 Output Screens with data	
4.3 Data Reports	
4.5 Sample program code	
4.6 Limitations and Bibliography	

CHAPTER 1 – INTRODUCTION

1. INTRODUCTION

WorkNest is a web-based organizational management system designed to manage users, roles, projects, and tasks in a structured and efficient manner. In many organizations, managing workflows manually or through disconnected tools leads to inefficiency, lack of transparency, and data inconsistency.

The WorkNest system provides a centralized platform where different users such as Admin, Manager, and Employee can interact based on their roles and permissions. The system ensures secure authentication, proper access control, and efficient task management.

The main objective of this system is to improve productivity, ensure accountability, and provide a scalable solution for managing organizational workflows.

1.1 Existing System and Need for System

Existing System

In the existing system, organizations rely on manual processes, spreadsheets, emails, or multiple tools to manage employees and projects.

Problems:

- No centralized system
- No proper access control
- Difficult to track tasks
- High chances of data errors

Need for System

The WorkNest system is required to:

- Provide centralized management
- Implement role-based access control
- Improve task tracking

- Increase productivity and transparency
-

1.2 Scope of Work

Current Scope

- User management
- Role-based access control
- Project creation
- Task assignment

Future Scope

- Notification system
 - Analytics dashboard
 - Integration with third-party tools
-

1.3 Operating Environment

Hardware Requirements

- Processor: Dual-core or above
- RAM: Minimum 2GB (4GB recommended)
- Storage: 20GB

Software Requirements

- OS: Windows/Linux
 - Frontend: React.js
 - Backend: Node.js (Express)
 - Database: PostgreSQL
 - Browser: Chrome / Firefox
-

1.4 Technology Used

Frontend (React.js)

Used for building user interface and handling user interactions.

Backend (Node.js)

Handles business logic, authentication, and APIs.

Database (PostgreSQL)

Stores structured data such as users, roles, projects, and tasks.

Security

- Authentication
 - Role-Based Access Control (RBAC)
-

CHAPTER 2 – PROPOSED SYSTEM

2. PROPOSED SYSTEM

The proposed system, WorkNest, is a web-based organizational management application designed to streamline user management, role allocation, project handling, and task assignment within an organization.

The system replaces manual and fragmented processes with a centralized platform that ensures better control, transparency, and efficiency. It follows a role-based access control mechanism where each user interacts with the system according to assigned permissions.

2.1 Proposed System

WorkNest follows a structured architecture where responsibilities are divided among different user roles such as Admin, Manager, and Employee.

The system provides the following functionalities:

- Centralized user and role management

- Secure authentication and authorization
- Project creation and management
- Task assignment and tracking
- Real-time workflow monitoring

This approach ensures better coordination and improved productivity within the organization.

2.1.1 Feasibility Study

Technical Feasibility

The WorkNest system is technically feasible because:

- It uses widely adopted technologies like React, Node.js, and PostgreSQL
 - Development tools are easily available
 - The system architecture is scalable and maintainable
 - Required technical knowledge is available
-

Economic Feasibility

The system is economically feasible because:

- It uses open-source technologies
 - Development cost is low
 - Reduces manual work and operational costs
 - Improves efficiency, saving time and resources
-

Operational Feasibility

The system is operationally feasible because:

- It has a simple and user-friendly interface
- Requires minimal training
- Easily adaptable in organizations
- Improves workflow management

2.2 Objectives of System

The main objectives of WorkNest are:

- To provide a centralized management system
- To implement secure role-based access control
- To improve project and task tracking
- To enhance productivity and accountability
- To ensure data security and consistency

2.3 User Requirements

The system must fulfill the following user requirements:

- Secure login and authentication
- Role-based access to features
- Easy project and task management
- Clear visibility of roles and responsibilities
- Ability to track progress
- Fast and reliable performance

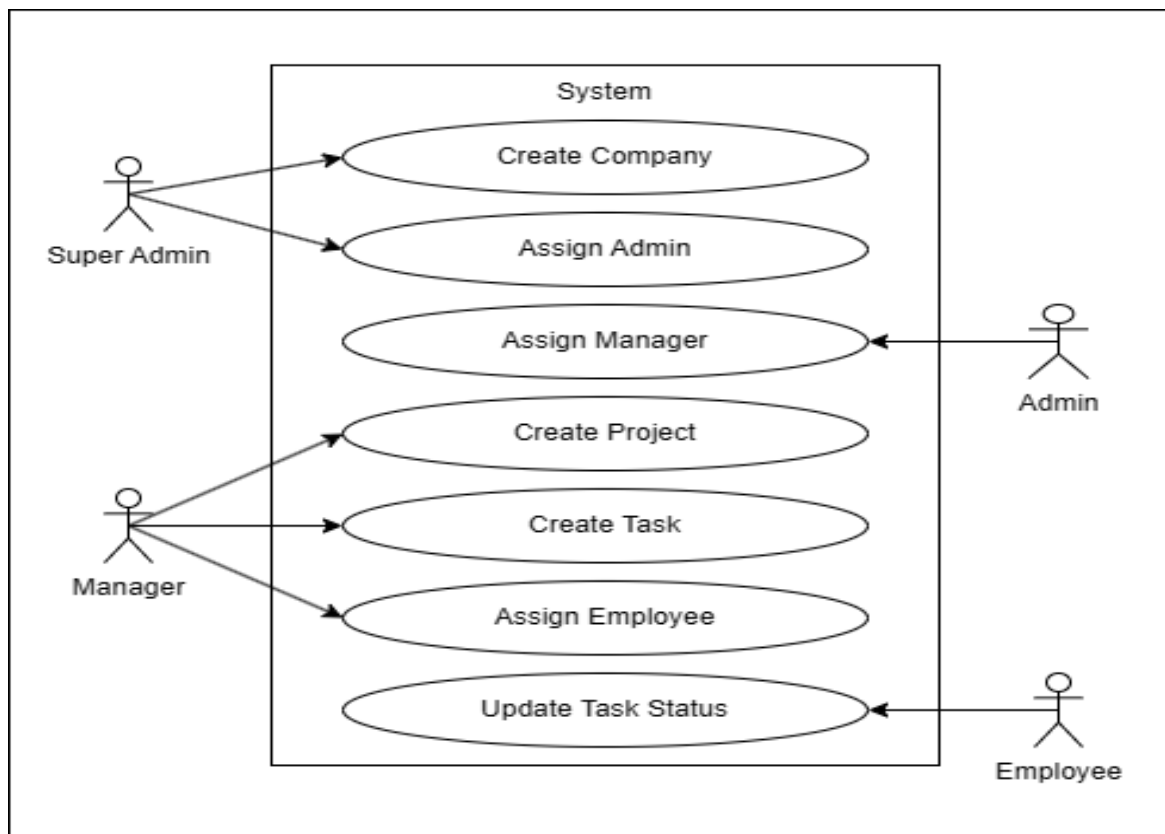
Meeting these requirements ensures that the system is effective, user-friendly, and efficient.

CHAPTER 3 – ANALYSIS AND DESIGN

3. ANALYSIS AND DESIGN

This chapter describes the system design, structure, and interactions between different components of the WorkNest system. It includes diagrams and database design that represent the system functionality and workflow.

3.1 Use Case Diagram



The Use Case Diagram represents the interaction between users and the system. It defines how different users interact with the system functionalities.

Actors:

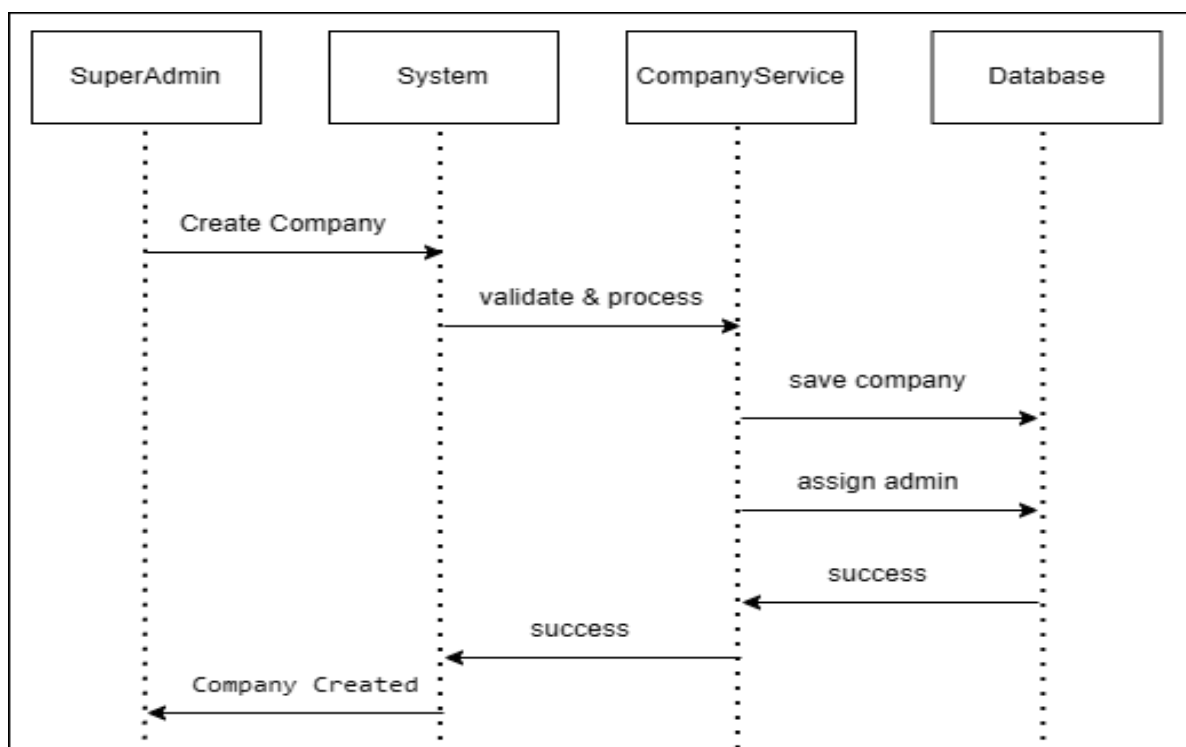
- Admin
- Super Admin
- Manager

- Employee

Use Cases:

- Login / Logout
- Manage Users
- Create Projects
- Assign Tasks
- Update Task Status

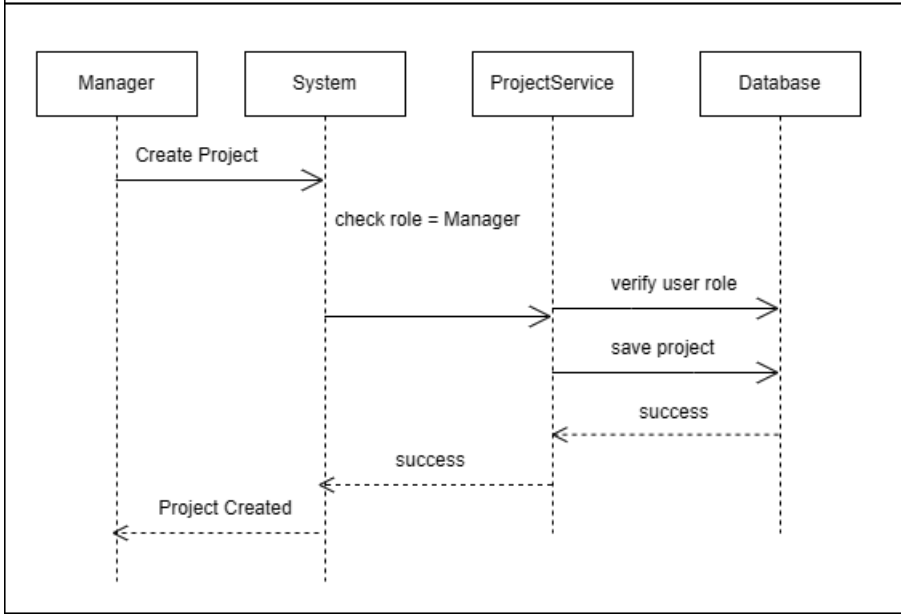
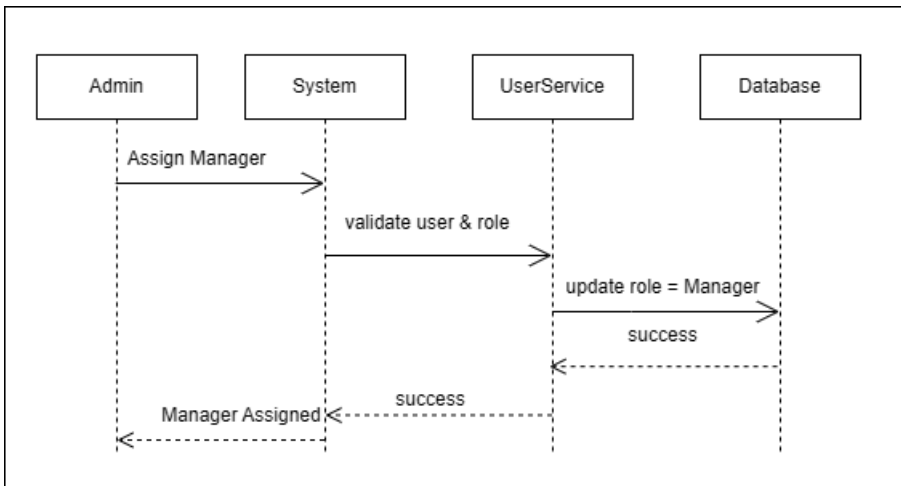
3.2 Sequence Diagram

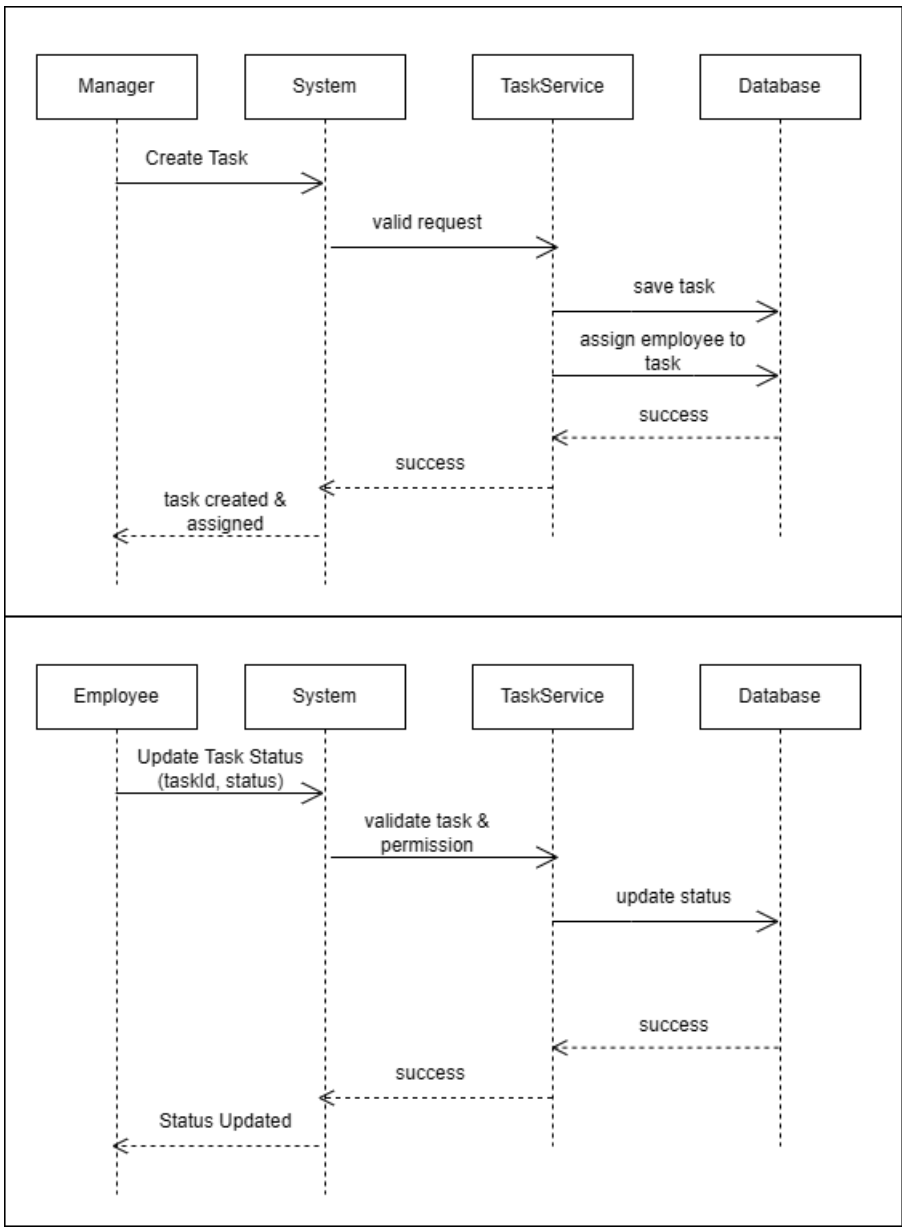


The Sequence Diagram shows the step-by-step interaction between system components during user operations.

Flow:

1. User enters credentials
2. Frontend sends request to backend
3. Backend validates data
4. Database verifies user
5. System sends response
6. User gets access



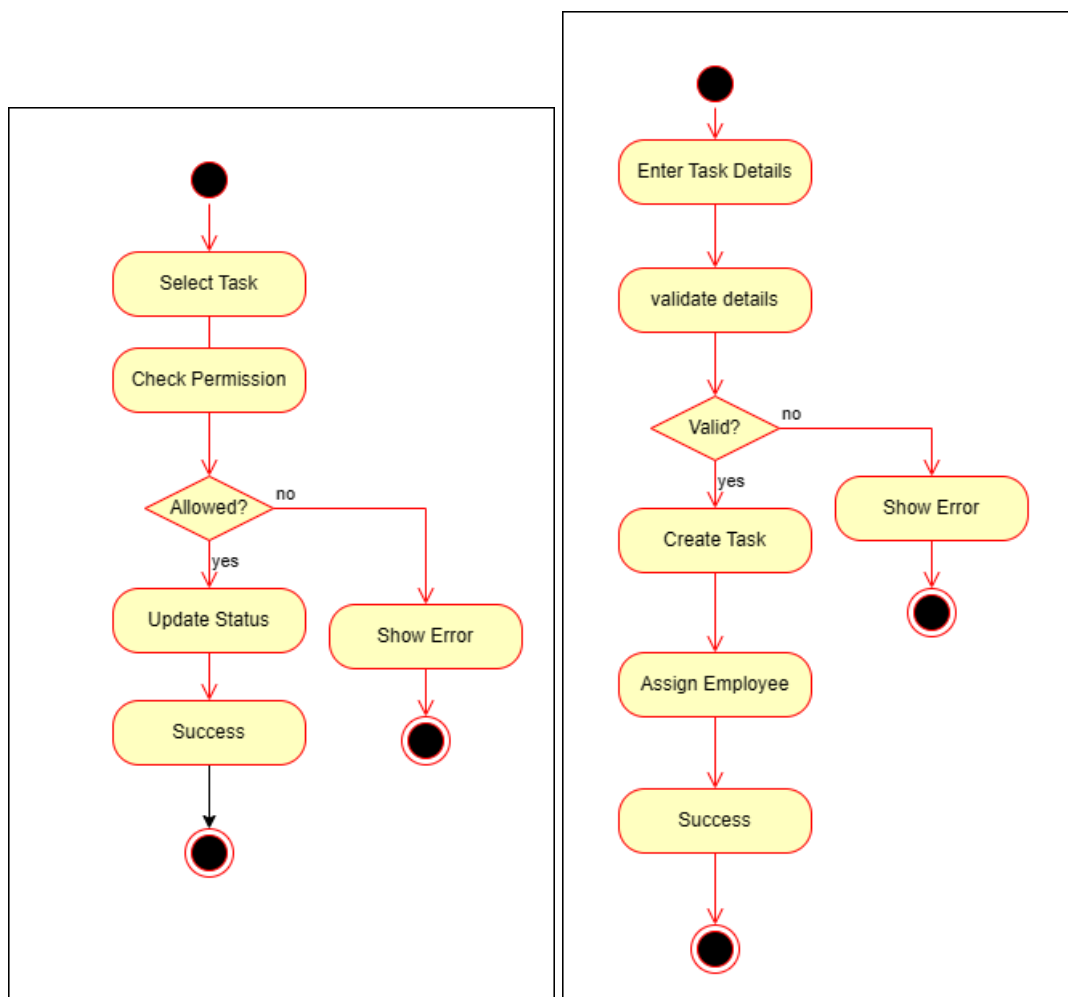


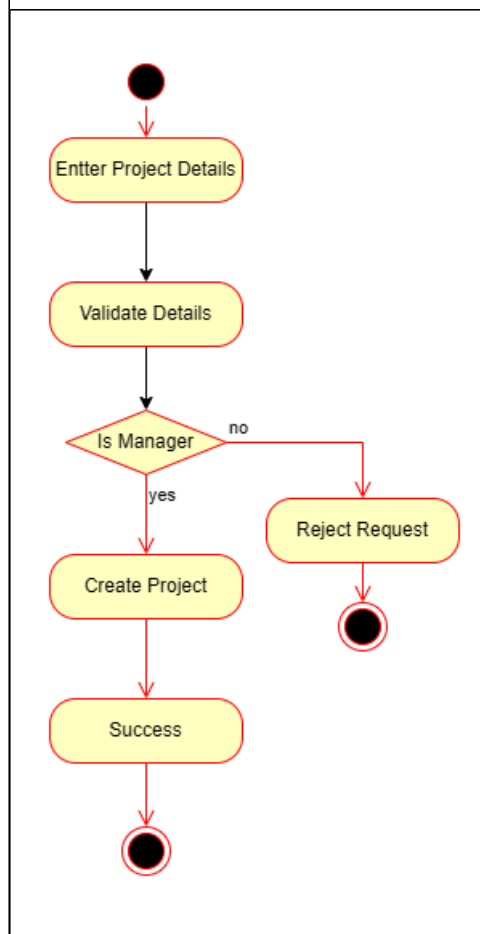
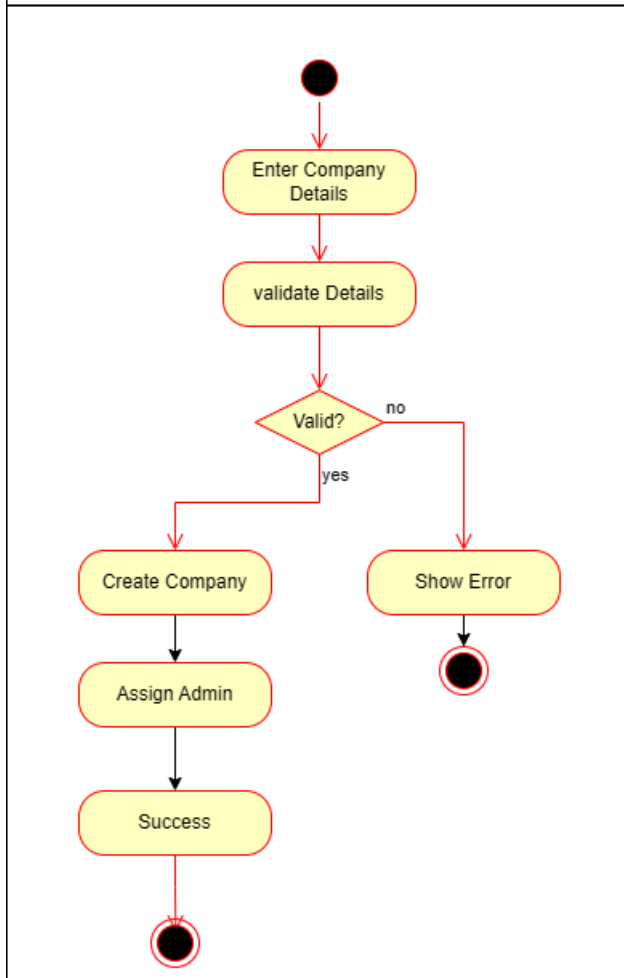
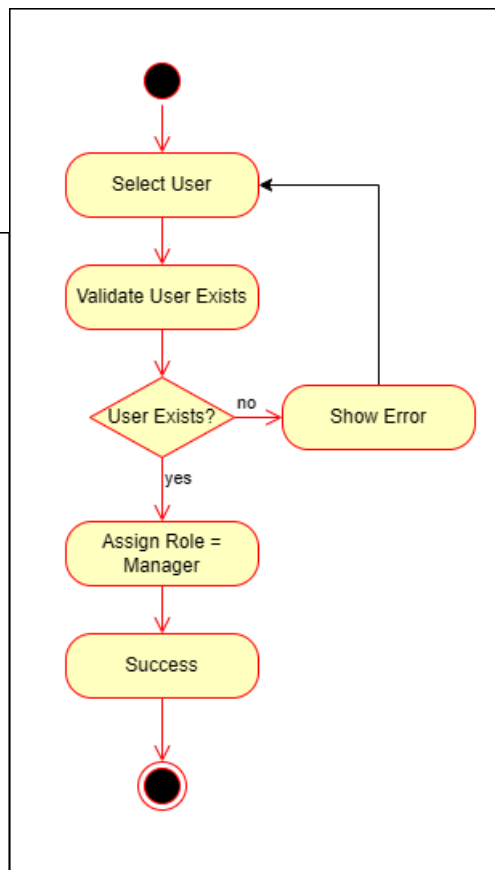
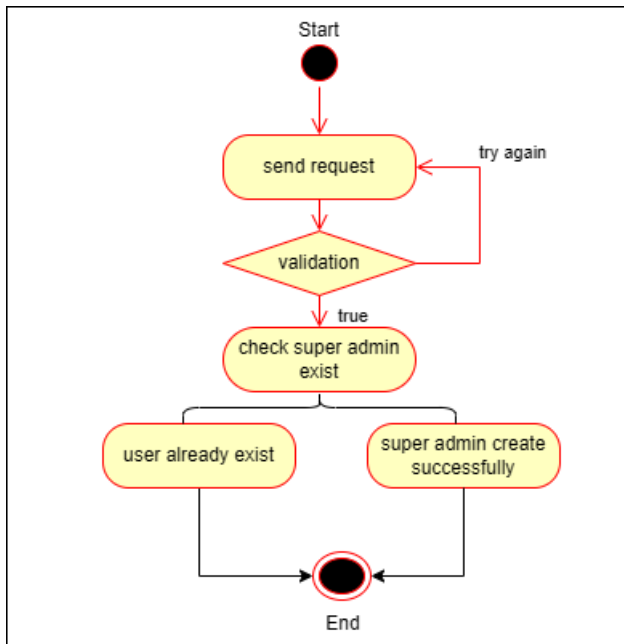
3.3 Activity Diagram

The Activity Diagram represents the workflow of the system by showing the sequence of activities performed by the user and system. It helps in understanding the process flow from login to task management.

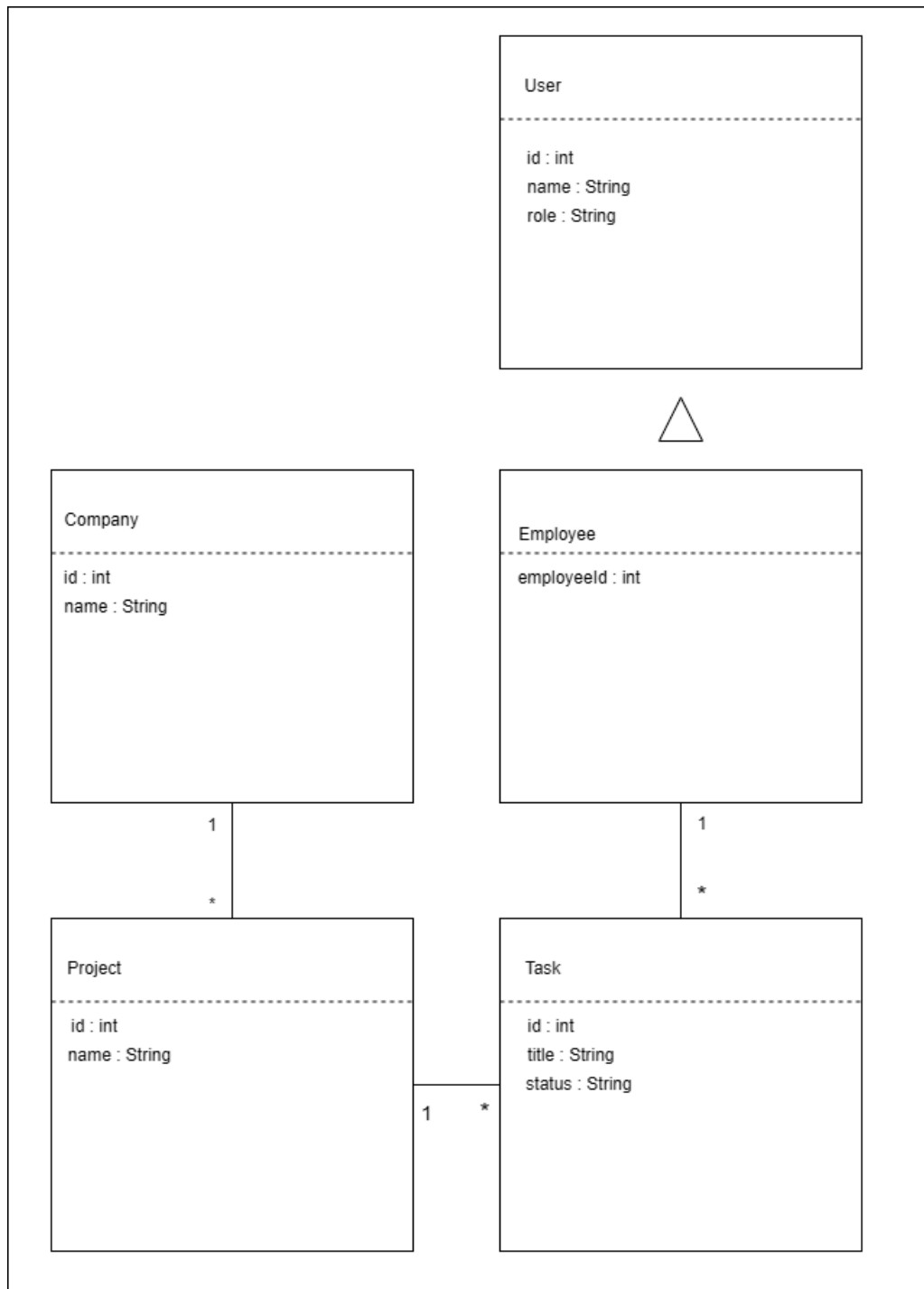
Flow Description:

1. User opens the application
2. User enters login credentials
3. System validates credentials
4. If valid → user is redirected to dashboard
5. If invalid → error message is shown
6. User selects operation (Create Project / Assign Task)
7. System processes request
8. Data is stored in database
9. System displays updated information





3.4 Class Diagram



The Class Diagram represents the structure of the system including classes, attributes, and relationships.

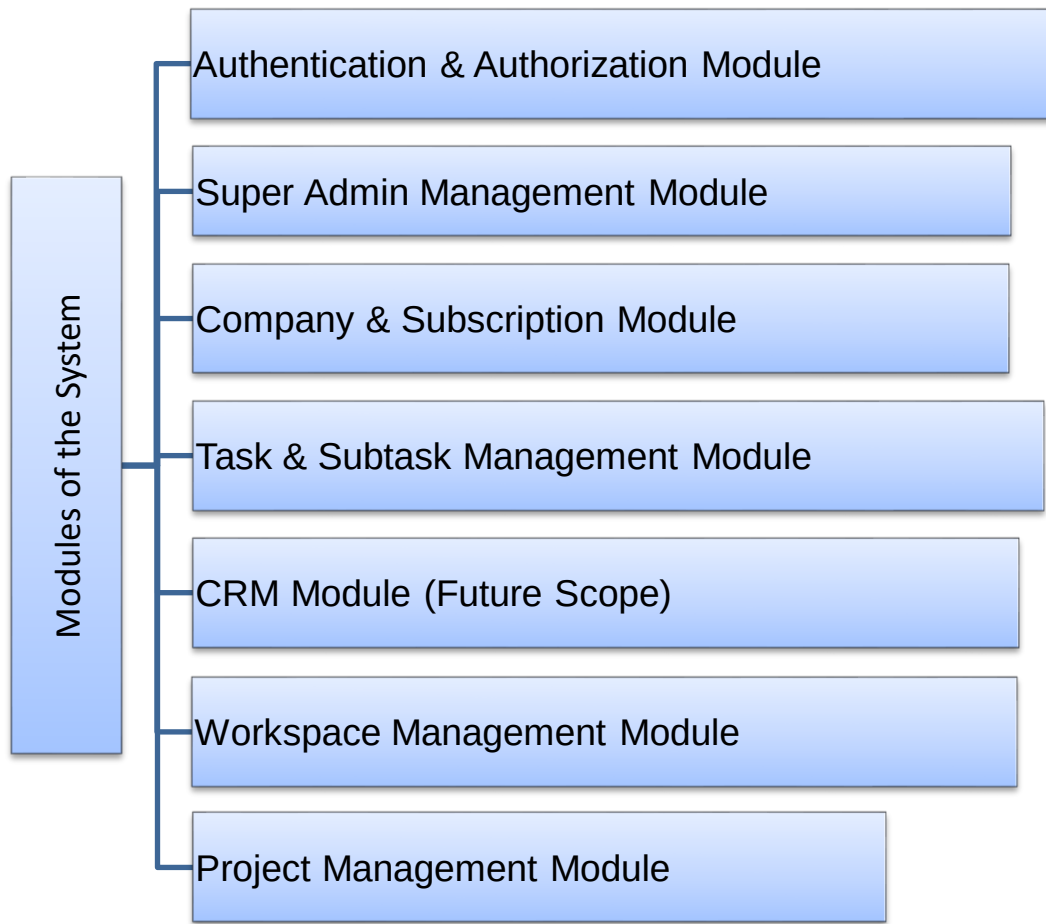
Main Classes:

- User
- Company
- Role
- Project
- Task

Short explain:

- User → stores user details
 - Role → defines permissions
 - Project → manages projects
 - Task → handles tasks
-

3.5 Module Hierarchy Diagram



Structure:

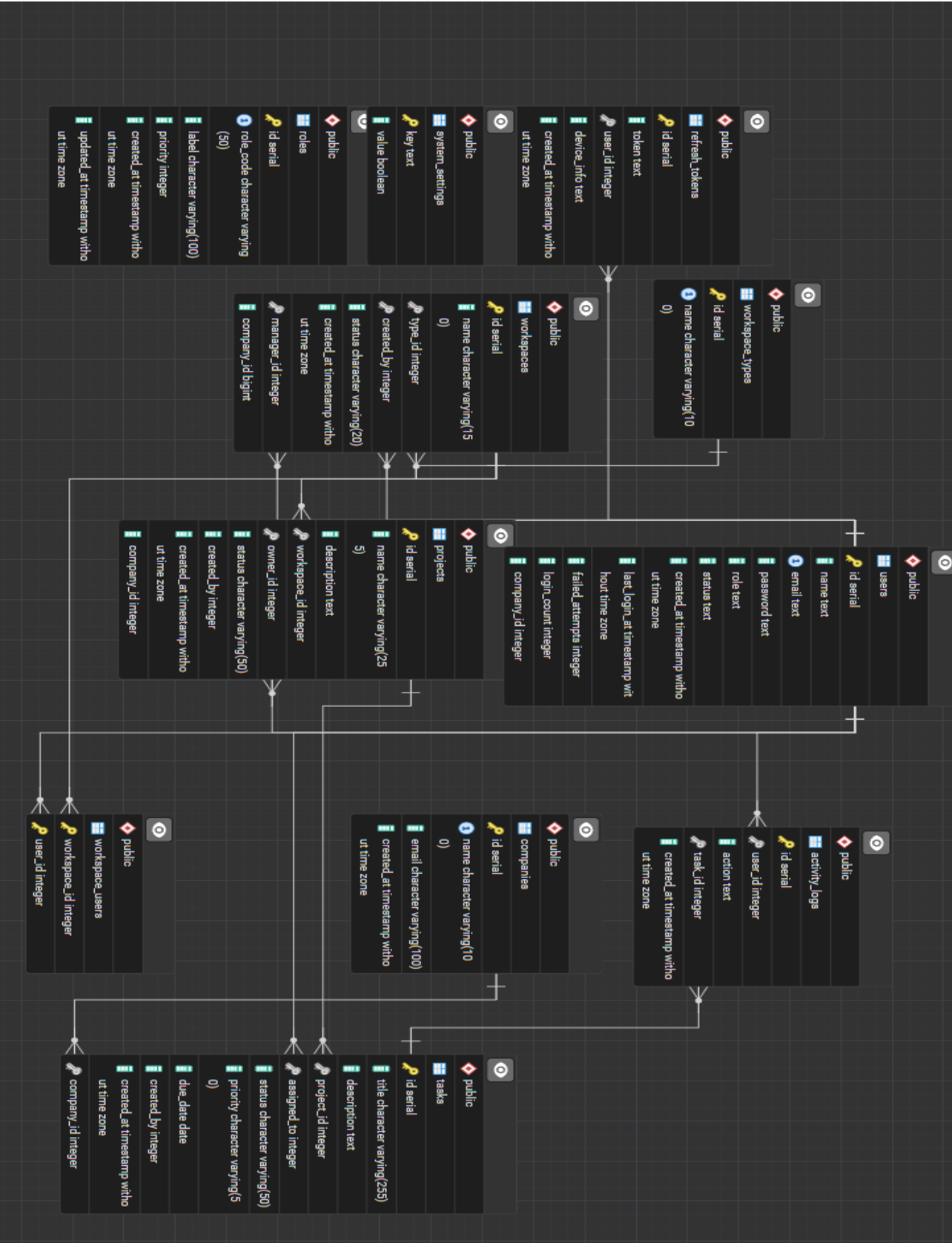
WorkNest System

- └─ Authentication Module
- └─ User Management Module
- └─ Project Management Module
- └─ Task Management Module

Description:

Each module is responsible for specific functionality such as authentication, user handling, and task processing.

3.6 DATABASE DESIGN (VERY IMPORTANT)



Tables:

1. Users Table

- id (Primary Key)
 - name
 - email
 - password
 - role_id (Foreign Key)
-

2. Roles Table

- id
 - role_name
-

3. Projects Table

- id
 - name
 - manager_id (Foreign Key)
-

4. Tasks Table

- id
 - title
 - description
 - project_id (Foreign Key)
 - assigned_to (Foreign Key)
-

Relationships:

- One Role → Many Users
 - One User → Many Tasks
 - One Project → Many Tasks
-

3.7 Data Dictionary

The Data Dictionary defines the meaning of database fields used in the system.

Example:

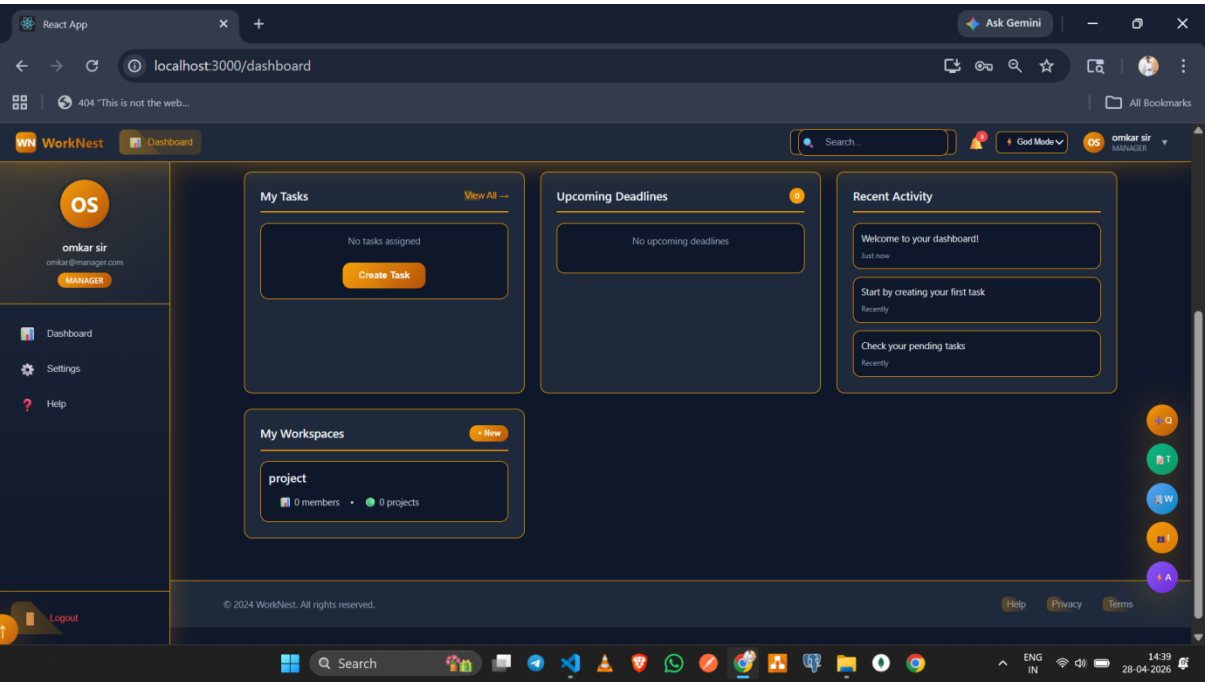
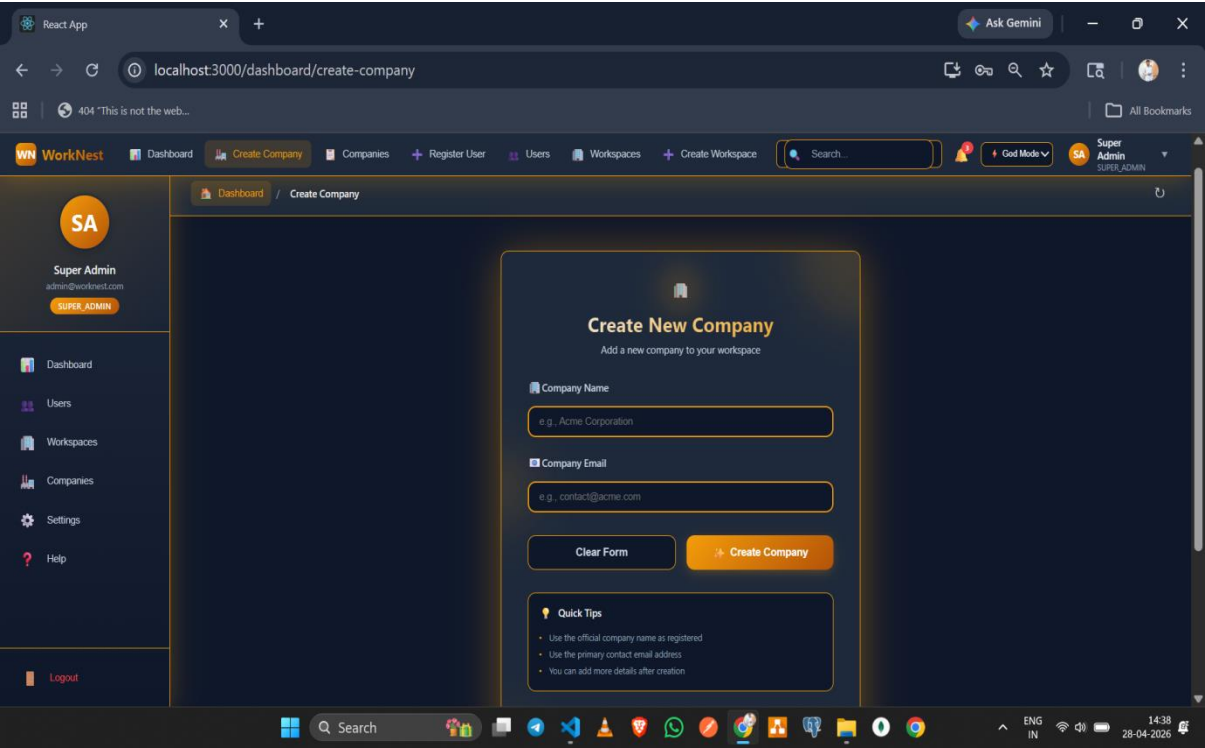
- id → Unique identifier (Primary Key)
 - role_id → Links user with role
 - project_id → Identifies project
 - assigned_to → Task assigned user
-

CHAPTER 4 – USER MANUAL

4. USER MANUAL

This chapter describes how users interact with the WorkNest system. It includes system screens, outputs, reports, and sample code used in the project.

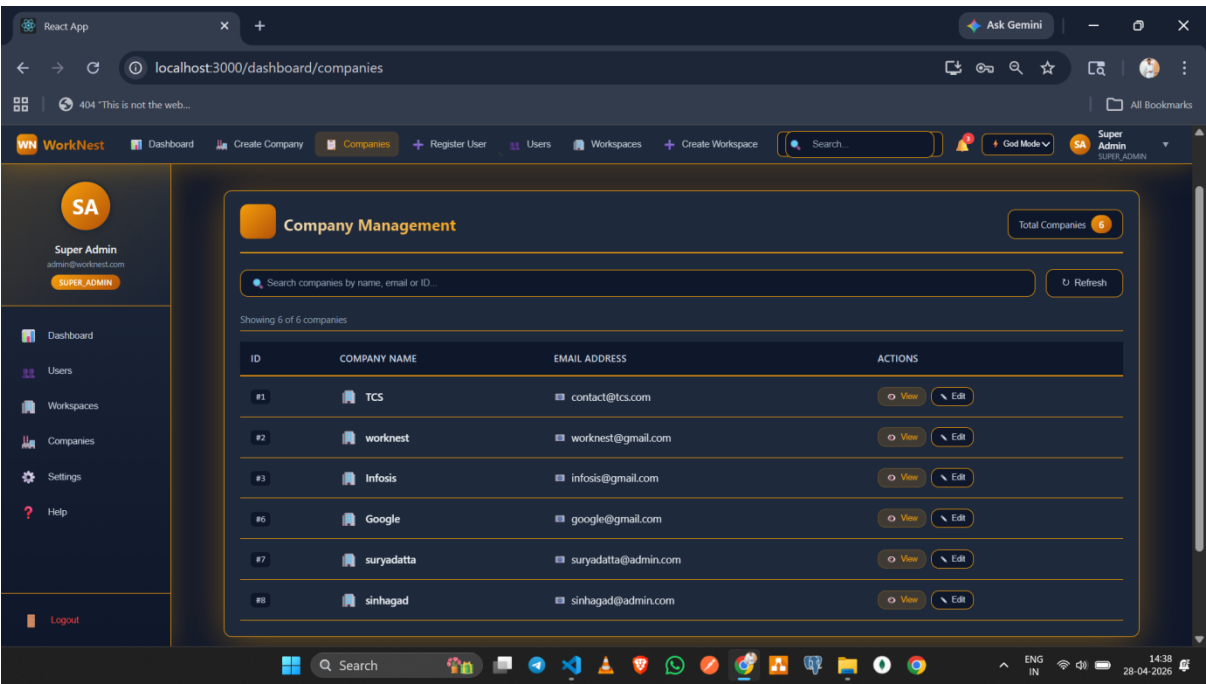
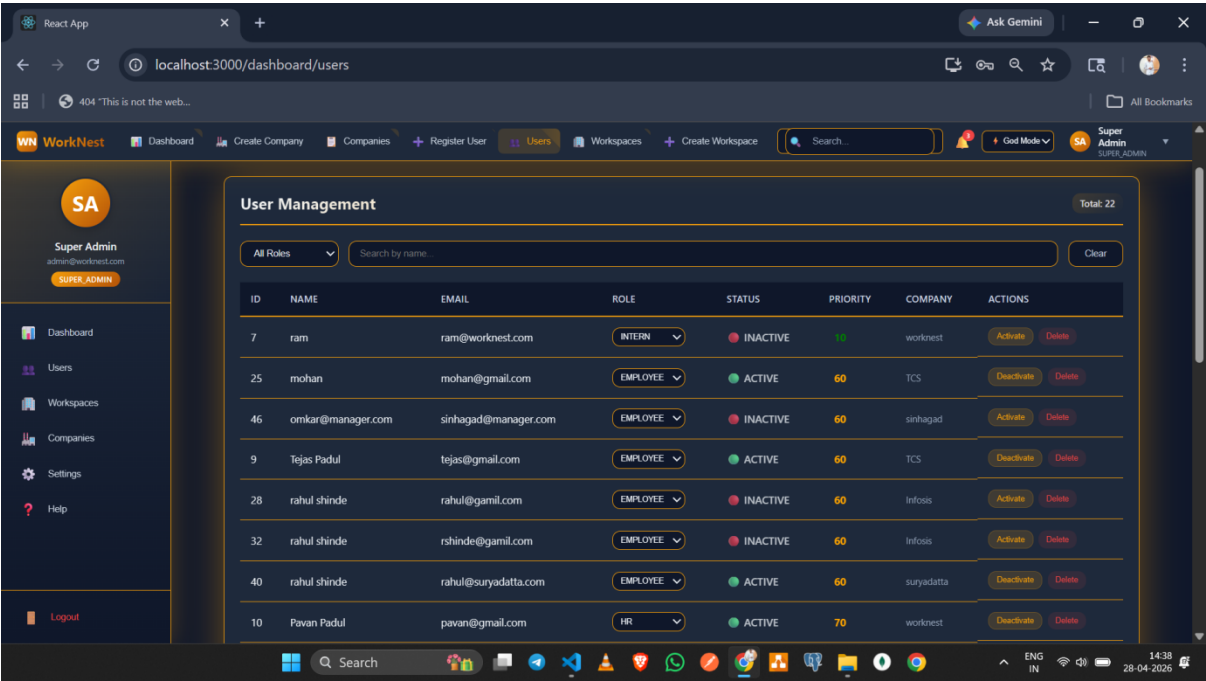
4.1 Input Screen

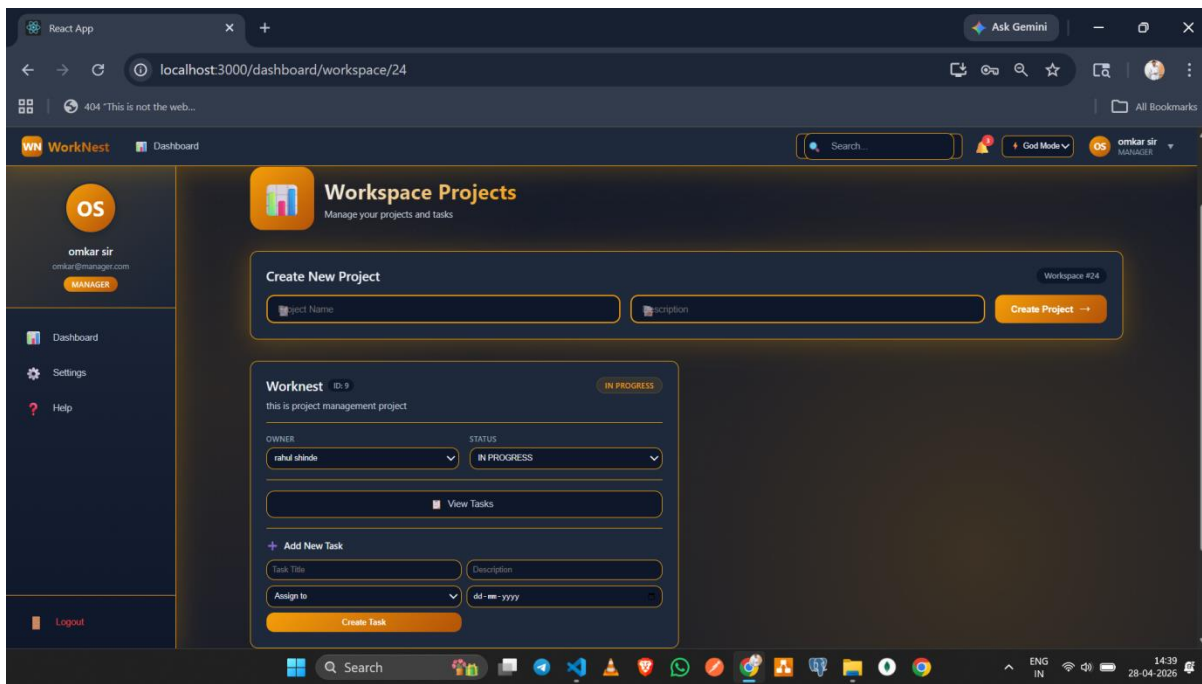


The system includes input screens such as Login Page, Registration Page, Create Project Form, and Task Assignment Form.

The input screens allow users to enter data into the system. These screens are designed to be simple, user-friendly, and validated to prevent errors.

4.2 Output Screens





- Dashboard
- Task List
- Project List

The output screens display processed data and system results. These screens provide users with insights into tasks, projects, and overall workflow.

4.3 Reports

The system provides basic reports such as:

- User Report
- Project Report
- Task Report

These reports help in monitoring system activities and tracking performance.

4.4 Sample Program Code

Login API (Node.js)

```
// ===== AUTH CONTROLLER =====  
// HTTP handling only  
  
const LoginDto = require('./dto/login.dto');  
const AuthService = require('./auth.service');  
const jwt = require("jsonwebtoken");  
  
class AuthController {  
  
  // ===== LOGIN =====  
  async login(req, res) {  
    return res.status(200).json({  
      success: true,  
      accessToken: result.accessToken,  
      user: result.user  
    });  
  
  }  
  
  // ===== REGISTER =====  
  async registerUser(req, res) {  
    try {  
      return res.status(201).json({  
        success: true,  
        message: 'Register user controller working'  
      });  
  
    } catch (err) {  
      console.error('REGISTER CONTROLLER ERROR:', err);  
  
      return res.status(400).json({  
        success: false,  
        message: err.message || 'Registration failed'  
      });  
    }  
  }  
}  
module.exports = new AuthController();
```

Middleware (Auth)

```
const jwt = require("jsonwebtoken");

function authMiddleware(req, res, next) {
  try {
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith("Bearer ")) {
      return res.status(401).json({ message: "Token missing or malformed" });
    }

    const token = authHeader.split(" ")[1];

    const decoded = jwt.verify(token, process.env.JWT_ACCESS_SECRET);

    req.user = {
      userId: decoded.userId,
      role: decoded.role,
      priority: decoded.priority,
      companyId: decoded.companyId // ❏ VERY IMPORTANT
    };

    next();
  } catch (error) {
    return res.status(401).json({ message: "Invalid or expired token" });
  }
}

module.exports = authMiddleware;
```

4.5 Limitations

- Internet connection required
- Basic reporting features
- Limited scalability in current version